MVC
Introducción y conceptos básicos

Concepto fundamental en el desarrollo de aplicaciones web: el **patrón MVC**, que son las siglas en inglés de **Model-View-Controller** (Modelo-Vista-Controlador). Este es un patrón de arquitectura de software que nos ayuda a organizar el código de nuestras aplicaciones de una forma **estructurada y mantenible**. **MVC** es ampliamente utilizado para organizar aplicaciones web. Divide la lógica de la aplicación en tres componentes principales:

¿Qué es MVC?

Imaginemos que estamos construyendo una casa. Necesitamos dividir el trabajo para que no todo quede en una sola persona. Al igual que en una casa, donde diferentes expertos se encargan de diferentes tareas, en una aplicación web usamos **MVC** para dividir nuestras responsabilidades. El patrón separa nuestra aplicación en tres componentes principales:

1. **Modelo (Model)**: El modelo es responsable de la lógica de negocio y la gestión de los datos. En una aplicación web, los modelos interactúan con la base de datos y contienen las reglas de negocio. Un modelo puede ser visto como la representación de los datos y las operaciones que se pueden realizar sobre ellos. Si pensamos en una aplicación de usuarios, el modelo sería la parte que sabe cómo crear, modificar o eliminar usuarios en la base de datos.
2. **Vista (View)**: La vista es la capa de presentación que interactúa directamente con el usuario. Esta parte no sabe nada sobre la base de datos o la lógica de negocio. Solo se preocupa por cómo mostrar los datos. En una aplicación web, las vistas suelen estar compuestas por HTML, CSS y JavaScript, lo que hace que la página sea atractiva y funcional para el usuario final.
3. **Controlador (Controller)**: El controlador actúa como un intermediario entre el modelo y la vista. Su tarea es recibir las solicitudes del usuario (por ejemplo, cuando un usuario hace clic en un enlace o envía un formulario), consultar o modificar los datos a través del modelo, y finalmente pasar esos datos a la vista para que sean presentados al usuario. En resumen, el controlador coordina las acciones entre el modelo y la vista.

Ejemplo de la vida real:

Imaginen que visitan un sitio web para ver su perfil. El proceso sería algo así:

1. Ustedes ingresan a una página como `miaplicacion.com/perfil`.
2. El **controlador** recibe la solicitud, sabe que están pidiendo el perfil, y llama al **modelo** para obtener los datos del usuario desde la base de datos.
3. Luego, el **modelo** entrega esos datos al controlador.
4. Finalmente, el **controlador** le pasa los datos a la **vista**, que los organiza en un formato bonito y muestra la información en la pantalla para que ustedes puedan verla.

¿Por qué usar MVC?

MVC nos ayuda a separar la lógica de la aplicación en componentes, lo que hace que nuestro código sea:

- **Más fácil de mantener**: Cada parte de la aplicación está separada, lo que facilita su modificación o actualización sin afectar otras partes.
- **Reutilizable**: Como cada componente está separado, podemos reutilizar modelos o vistas en diferentes partes de la aplicación.
- **Escalable**: Nos permite agregar nuevas funcionalidades de forma más ordenada, manteniendo un código limpio y organizado.

El patrón **MVC** es fundamental en el desarrollo web moderno porque nos permite organizar nuestras aplicaciones de forma eficiente, asegurando que podamos agregar nuevas funcionalidades sin que todo se desorganice. Cada componente tiene una responsabilidad específica, y juntos trabajan para entregar una experiencia fluida y mantenible para los desarrolladores y usuarios.

¿Cómo funciona el flujo de MVC?

1. **Solicitud del usuario**: Todo comienza cuando el usuario realiza una solicitud a través del navegador. Por ejemplo, al ingresar a `miaplicacion.com/productos/lista`, el navegador envía una solicitud HTTP al servidor.
2. **Enrutamiento**: En el servidor, la aplicación recibe esta solicitud. Aquí es donde entra en juego el **controlador**. En aplicaciones MVC, existe una capa de enrutamiento que mapea la URL solicitada al controlador y método correspondientes. En nuestro ejemplo, la URL `productos/lista` podría hacer referencia a un método `actionLista` en un controlador `ProductosController`.
3. **Interacción con el Modelo**: Una vez que el controlador recibe la solicitud, su trabajo es manejarla. Por ejemplo, si el usuario quiere ver la lista de productos, el controlador llamará al **modelo** para obtener los datos necesarios de la base de datos.
4. **Renderización de la Vista**: El modelo devuelve los datos al controlador, quien se los pasa a la **vista**. La vista toma esos datos y los organiza de manera que el usuario los vea de forma clara y atractiva.
5. **Respuesta al usuario**: Finalmente, la vista genera el HTML (u otro formato, como JSON si estamos trabajando con APIs) que se devuelve al navegador del usuario para ser mostrado.

¿Por qué MVC es útil en proyectos grandes?

A medida que las aplicaciones crecen, es crucial tener una arquitectura clara y bien organizada para evitar que el código se convierta en un caos difícil de mantener. MVC nos permite **separar las responsabilidades** en la aplicación:

- **Escalabilidad**: Como cada componente tiene una responsabilidad específica, es más fácil modificar o extender una parte de la aplicación sin afectar las otras. Por ejemplo, puedes cambiar la base de datos (modificando los modelos) sin alterar las vistas.
- **Mantenimiento**: Al tener el código dividido en capas (modelo, vista, controlador), es más fácil encontrar y corregir errores. Si algo no funciona bien en la presentación, sabemos que probablemente sea un problema en la vista. Si hay un problema con los datos, el modelo será el principal sospechoso.
- **Reutilización de código**: Los modelos y controladores suelen ser reutilizables. Puedes utilizar el mismo modelo para diferentes controladores o vistas, y tener controladores que respondan tanto con vistas HTML como con APIs RESTful en JSON.

Desglose más profundo del patrón:

1. El Modelo: más que solo acceso a datos

Es importante entender que el **modelo** no solo se limita a interactuar con la base de datos. También contiene las **reglas de negocio**. Un modelo bien diseñado debería ser capaz de manejar toda la lógica que envuelve a los datos, como:

- Validaciones: Por ejemplo, asegurarse de que el correo electrónico de un usuario es válido antes de guardarlo.
- Relaciones entre datos: Por ejemplo, en una aplicación de ventas, el modelo de un producto puede estar relacionado con los modelos de categorías, proveedores o inventarios.
- Cálculos: Si necesitas realizar operaciones matemáticas (como calcular el precio total de un pedido), el modelo es el lugar adecuado para hacerlo.

Ejemplo: Supongamos que tenemos un modelo `Usuario`. Este modelo debería ser capaz de validar que el correo electrónico de un usuario es único, de cifrar su contraseña antes de guardarla, y de gestionar su relación con otros modelos, como el de roles o permisos.

```php
class UsuarioModel extends Model {
    protected $table = 'usuarios';

    public function registrar($data) {
        // Validaciones
        if (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
            throw new Exception('Correo electrónico inválido');
        }
        // Cifrar la contraseña antes de guardarla
        $data['password'] = password_hash($data['password'], PASSWORD_BCRYPT);

        // Guardar el usuario en la base de datos
        return $this->insert($data);
    }
}
```

2. La Vista: dinámica y reutilizable

Las **vistas** son lo que el usuario final ve. Aquí es donde se muestra el HTML generado. En aplicaciones más complejas, las vistas suelen estar divididas en **componentes** o **templates** reutilizables.

Es común tener vistas dinámicas que se basan en los datos enviados por el controlador. Las vistas en MVC no deberían contener lógica compleja, solo se encargan de presentar los datos.

Ejemplo: Supongamos que tienes una vista que muestra la lista de productos en una tienda online.

```
<!-- archivo productos.php -->
<h1>Lista de Productos</h1>
<ul>
<?php foreach ($productos as $producto): ?>
    <li><?php echo $producto['nombre']; ?> - <?php echo $producto['precio']; ?> USD</li>
<?php endforeach; ?>
</ul>
```

En este caso, el controlador es el encargado de pasar la lista de productos como un array a la vista, que simplemente los muestra.

3. El Controlador: el intermediario

El **controlador** maneja la lógica de flujo. Recibe las solicitudes del usuario, interactúa con los modelos, y selecciona la vista adecuada para mostrar la respuesta.

Un buen controlador debe ser **ligero**. La mayor parte de la lógica de negocio debería residir en los modelos. El controlador debe centrarse en **coordinar** las interacciones entre los modelos y las vistas.

Ejemplo: Un controlador para manejar las solicitudes a la lista de productos.

```
class ProductosController extends Controller {
    public function actionLista() {
        $productos = ProductoModel::getAll();  // Obtener todos los productos desde el modelo
        $this->render('productos', ['productos' => $productos]);  // Pasar los productos a la vista
    }
}
```

En este ejemplo, el controlador es responsable de:

1. Recibir la solicitud para la lista de productos.
2. Obtener los datos necesarios del modelo `ProductoModel`.
3. Renderizar la vista `productos.php` pasándole los datos.

Buenas prácticas en MVC

1. **Mantener el controlador ligero**: El controlador debería hacer solo lo mínimo necesario. Su trabajo es coordinar la interacción entre el modelo y la vista, pero no debería contener lógica de negocio. Esa lógica debe estar en el modelo.
2. **Reutilizar vistas**: Intenta crear componentes reutilizables en las vistas, como cabezales, menús, o pies de página. Esto ayuda a mantener el código más organizado y permite cambios rápidos en la interfaz.
3. **Lógica de negocio en el modelo**: Asegúrate de que toda la lógica relacionada con los datos esté en el modelo. Por ejemplo, no querrás calcular precios o aplicar impuestos directamente en el controlador o la vista.

¿Qué es el enrutamiento?

El enrutamiento es el mecanismo que permite a tu aplicación determinar qué controlador y qué método (o acción) deben manejar la solicitud del usuario basándose en la URL.

Por ejemplo, si el usuario navega a la URL:

```
https://miaplicacion.com/productos/ver/23
```

En este caso:

- `productos` indica el controlador a usar, en este caso, `ProductosController`.
- `ver` indica el método o acción, en este caso `actionVer()`.
- `23` es el parámetro, que podría ser el ID del producto.